

Lesson 12 — Complete Polynomial Data-Structure Program

1. Where We Are Now

You are at the final lesson of this curriculum.

Before Lesson 12, you already learned these ideas:

Lesson	Main idea
Lesson 1	Store only useful matrix values instead of storing many zeros.
Lesson 2	Store sparse matrix non-zero values using row, column, and value.
Lesson 3	Display a sparse matrix as a full matrix.
Lesson 4	Add sparse matrices using a merge-style algorithm.
Lesson 5	Write a sparse matrix addition function.
Lesson 6	Complete sparse matrix addition in C using <code>malloc</code> and <code>free</code> .
Lesson 7	Convert sparse matrix code into a C++ class.
Lesson 8	Use operator overloading for sparse matrices in C++.
Lesson 9	Represent a polynomial using terms.
Lesson 10	Evaluate a polynomial for a given value of <code>x</code> .
Lesson 11	Add polynomials using a merge-style algorithm.
Lesson 12	Write the complete C program for polynomial addition.

Lesson 12 is not a completely new idea. It is the complete C program version of the polynomial addition algorithm from Lesson 11.

The important idea is:

```
Same exponent      → add coefficients
Different exponent → copy the term with the larger exponent
```

2. Main Goal of Lesson 12

The goal of Lesson 12 is to build a complete C program that can:

1. Create the first polynomial.
2. Create the second polynomial.
3. Display the first polynomial.
4. Display the second polynomial.
5. Add the two polynomials.
6. Display the sum polynomial.
7. Use dynamic memory with `malloc`.
8. Release memory with `free`.

The complete program uses these main parts:

```
struct term
struct Poly
create()
display()
add()
main()
```

3. What Is a Polynomial?

A polynomial is an expression made by adding terms.

Example:

```
3x^5 + 2x^4 + 5x^2 + 2x + 7
```

This polynomial has 5 terms:

```
3x^5
2x^4
5x^2
2x
7
```

Each term has two important parts:

Part	Meaning	Example in <code>3x^5</code>
Coefficient	The number multiplied by <code>x</code>	<code>3</code>
Exponent	The power of <code>x</code>	<code>5</code>

So:

```
3x^5
```

is stored as:

```
coefficient = 3
exponent    = 5
```

4. Why We Store a Polynomial as Terms

A beginner may think we should store a polynomial as a string.

Example:

```
char p[] = "3x^5 + 2x^4 + 5x^2 + 2x + 7";
```

But this is not useful for data-structure operations.

Why?

Because later we need to:

- compare exponents,
- add coefficients,
- copy terms,
- evaluate terms,
- combine like terms.

These operations are easier if each term is stored separately.

So instead of storing the symbols:

x
+
^

we store only:

coefficient, exponent

Example polynomial:

$3x^5 + 2x^4 + 5x^2 + 2x + 7$

Stored form:

Index	Coefficient	Exponent	Term
0	3	5	$3x^5$
1	2	4	$2x^4$
2	5	2	$5x^2$
3	2	1	$2x^1$
4	7	0	$7x^0$

5. Important Polynomial Rules

Rule 1: If a coefficient is written, use that coefficient

Example:

$7x^3$

This means:

coefficient = 7
exponent = 3

Rule 2: If no coefficient is written, the coefficient is 1

Example:

x^4

This actually means:

$1x^4$

So we store:

```
coefficient = 1  
exponent   = 4
```

Another example:

$-x^4$

This means:

$-1x^4$

So we store:

```
coefficient = -1  
exponent    = 4
```

Rule 3: If x has no written exponent, the exponent is 1

Example:

$2x$

This actually means:

$$2x^1$$

So we store:

```
coefficient = 2  
exponent   = 1
```

Rule 4: If there is no x , the exponent is 0

Example:

$$7$$

This actually means:

$$7x^0$$

Why?

Because:

$$x^0 = 1$$

So:

$$7x^0 = 7 \times 1 = 7$$

Therefore, the constant term 7 is stored as:

```
coefficient = 7  
exponent    = 0
```

6. The term Structure

One term needs two values:

coefficient
exponent

So we create a structure called `term`.

```
struct term {  
    int coeff;  
    int exp;  
};
```

Meaning:

Field	Meaning
<code>coeff</code>	coefficient
<code>exp</code>	exponent

Example:

```
struct term t;  
t.coeff = 3;  
t.exp = 5;
```

This represents:

$3x^5$

One `term` does not store the whole polynomial.

It stores only one part of the polynomial.

7. The `Poly` Structure

A polynomial usually has many terms.

So we need another structure to store the full polynomial.

```
struct Poly {  
    int n;
```

```
struct term *terms;
};
```

Meaning:

Field	Meaning
<code>n</code>	number of terms in the polynomial
<code>terms</code>	pointer to a dynamic array of terms

Example:

For this polynomial:

$$3x^5 + 2x^4 + 5x^2 + 2x + 7$$

There are 5 terms.

So:

$$n = 5$$

The pointer `terms` points to an array of 5 `struct term` records.

Memory idea:

```
p
├── n = 5
└── terms ──> terms[0] = {3, 5}
               terms[1] = {2, 4}
               terms[2] = {5, 2}
               terms[3] = {2, 1}
               terms[4] = {7, 0}
```

8. Important Meaning of `n`

In sparse matrix lessons, `n` often meant:

number of columns

But in polynomial lessons, **n** means:

number of terms

This is very important.

Example:

$3x^5 + 7$

This polynomial has 2 terms:

$3x^5$
 7

So:

$n = 2$

Even though the highest exponent is 5, **n** is not 5.

Correct:

$n = \text{number of terms}$

Wrong:

$n = \text{highest exponent}$

9. Dynamic Memory in Lesson 12

Before the program runs, we do not know how many terms the user will enter.

The user may enter:

3 terms

or:

10 terms

or:

100 terms

So we should not create a fixed-size array blindly.

Instead, we first read `n`.

Then we allocate exactly enough memory.

```
p->terms = (struct term *) malloc(p->n * sizeof(struct term));
```

If:

```
p->n = 5
```

then this creates space for:

```
terms[0]  
terms[1]  
terms[2]  
terms[3]  
terms[4]
```

So the program stores only the actual terms.

10. Why We Use `malloc`

In C, we use `malloc` to create memory while the program is running.

This is called dynamic memory allocation.

General form:

```
pointer = (type *) malloc(number_of_items * sizeof(type));
```

For polynomial terms:

```
p->terms = (struct term *) malloc(p->n * sizeof(struct term));
```

Meaning:

Create memory for p->n term records.

Example:

If `p->n = 4`, then:

```
p->terms = (struct term *) malloc(4 * sizeof(struct term));
```

This creates space for 4 terms.

11. Why We Use `free`

Memory created using `malloc` should be released using `free`.

If we allocate memory and never release it, the program may waste memory.

This is called a memory leak.

Memory rule:

Allocation	Deallocation
<code>malloc</code>	<code>free</code>
<code>calloc</code>	<code>free</code>
<code>new</code>	<code>delete</code>
<code>new[]</code>	<code>delete[]</code>

In Lesson 12, we use C style:

```
malloc → free
```

Example:

```
free(p1.terms);  
free(p2.terms);  
free(p3->terms);  
free(p3);
```

12. Dot Operator `.` and Arrow Operator `->`

This lesson uses both normal structures and pointers to structures.

That means we must know when to use `.` and when to use `->`.

Use `.` when you have the actual structure variable

Example:

```
struct Poly p1;
```

Here, `p1` is the actual structure variable.

So we write:

```
p1.n  
p1.terms
```

Use `->` when you have a pointer to a structure

Example:

```
struct Poly *p;
```

Here, `p` is a pointer.

So we write:

```
p->n  
p->terms
```

Simple rule:

Situation	Operator
Normal structure variable	.
Pointer to structure	->

13. The `create()` Function

The `create()` function reads one polynomial from the user.

It does these steps:

1. Reads the number of terms.
2. Allocates memory for the terms.
3. Reads each coefficient and exponent.
4. Stores each term in the dynamic array.

Code:

```
void create(struct Poly *p) {  
    int i;  
  
    printf("Enter number of terms: ");  
    scanf("%d", &p->n);  
  
    p->terms = (struct term *) malloc(p->n * sizeof(struct term));  
  
    if (p->terms == NULL && p->n > 0) {  
        printf("Memory allocation failed.\n");  
        exit(1);  
    }  
  
    printf("Enter coefficient and exponent for each term in descending exponent  
order:\n");  
  
    for (i = 0; i < p->n; i++) {  
        scanf("%d %d", &p->terms[i].coeff, &p->terms[i].exp);  
    }  
}
```

```
}  
}
```

14. Why `create()` Receives a Pointer

The function header is:

```
void create(struct Poly *p)
```

Here, `p` is a pointer to a `struct Poly`.

We pass a pointer because `create()` must change the original polynomial from `main()`.

In `main()`, we call:

```
create(&p1);
```

The symbol `&` means address.

So:

```
&p1
```

means:

```
the address of p1
```

Because `create()` receives the address, it can modify the original `p1`.

It can fill:

```
p1.n  
p1.terms  
p1.terms[0]  
p1.terms[1]  
...
```

If we passed only a copy, the original polynomial in `main()` would not be properly created.

15. Why We Use `->` Inside `create()`

Inside `create()`, the variable `p` is a pointer.

So we use `->`:

```
p->n  
p->terms  
p->terms[i].coeff  
p->terms[i].exp
```

Do not write:

```
p.n  
p.terms
```

That would be wrong because `p` is not the actual structure.

It is a pointer to the structure.

16. Reading Terms in Descending Exponent Order

The polynomial terms should be entered in descending exponent order.

Descending order means:

```
highest exponent first  
lowest exponent last
```

Example of good order:

```
5x^4 + 2x^2 + 7x^0
```

Exponents:

```
4, 2, 0
```

This is descending order.

Example of bad order:

$$2x^2 + 5x^4 + 7x^0$$

Exponents:

2, 4, 0

This is not descending order.

Why order matters:

The addition algorithm moves forward only.

It does not go back and search earlier terms.

So if the terms are not sorted, the result may be wrong.

This is similar to sparse matrix addition:

Sparse matrix addition

Polynomial addition

Elements must be sorted by row and column.

Terms must be sorted by exponent.

17. Dry Run of `create()`

Suppose the polynomial is:

$$3x^5 + 2x^4 + 5x^2 + 2x + 7$$

The user enters:

5
3 5
2 4
5 2


```
2 1
7 0
```

Step 1:

```
p->n = 5
```

Step 2:

```
p->terms = malloc(5 * sizeof(struct term));
```

Now memory is created for 5 terms.

Step 3: Read each term.

Loop index	Input	Stored result	Meaning
0	3 5	terms[0] = {3, 5}	$3x^5$
1	2 4	terms[1] = {2, 4}	$2x^4$
2	5 2	terms[2] = {5, 2}	$5x^2$
3	2 1	terms[3] = {2, 1}	$2x^1$
4	7 0	terms[4] = {7, 0}	$7x^0$

Final stored form:

```
terms[0] = {3, 5}
terms[1] = {2, 4}
terms[2] = {5, 2}
terms[3] = {2, 1}
terms[4] = {7, 0}
```

18. The `display()` Function

The `display()` function prints a polynomial.

Code:

```

void display(struct Poly p) {
    int i;

    if (p.n == 0) {
        printf("0\n");
        return;
    }

    for (i = 0; i < p.n; i++) {
        printf("%dx^%d", p.terms[i].coeff, p.terms[i].exp);

        if (i < p.n - 1) {
            printf(" + ");
        }
    }

    printf("\n");
}

```

19. Why `display()` Receives the Structure by Value

The function header is:

```
void display(struct Poly p)
```

Here, `p` is not a pointer.

It is a normal structure variable inside the function.

The display function only needs to read the polynomial and print it.

It does not need to change the original polynomial.

So passing by value is enough.

Because `p` is a normal structure variable, we use the dot operator:

```
p.n  
p.terms[i].coeff  
p.terms[i].exp
```

Do not use:

```
p->n  
p->terms
```

because `p` is not a pointer inside `display()`.

20. Why We Print `+` Only Between Terms

When displaying the polynomial, we want this:

```
3x^5 + 2x^4 + 7x^0
```

We do not want this:

```
3x^5 + 2x^4 + 7x^0 +
```

So the code prints `+` only if the current term is not the last term.

```
if (i < p.n - 1) {  
    printf(" + ");  
}
```

Meaning:

```
If this is not the final term, print plus.  
If this is the final term, do not print plus.
```

21. The Main Idea of Polynomial Addition

To add two polynomials, we combine like terms.

Like terms are terms with the same exponent.

Example:

$$5x^4 + 6x^4 = 11x^4$$

Why?

Because both terms have exponent .

Only the coefficients are added:

$$5 + 6 = 11$$

The exponent stays the same:

Another example:

$$2x^2 + 9x^2 = 11x^2$$

Again:

$$2 + 9 = 11$$

The exponent stays:

Terms with different exponents cannot be combined.

Example:

$$5x^4 + 2x^3$$

These are not like terms.

So we keep both:

```
5x^4 + 2x^3
```

22. Addition Rule

The rule is simple:

```
If exponents are equal:  
    Add coefficients.  
    Store one result term.  
  
If exponents are different:  
    Copy the term with the larger exponent.
```

Why copy the larger exponent first?

Because we want the result to stay in descending exponent order.

Example:

```
x^5, x^4, x^2, x^1, x^0
```

This order is easier to read and easier to process.

23. The `add()` Function

The `add()` function adds two polynomials and returns a pointer to the result polynomial.

Function header:

```
struct Poly *add(struct Poly *p1, struct Poly *p2)
```

Meaning:

Part	Meaning
<code>struct Poly *</code> before <code>add</code>	The function returns a pointer to a <code>struct Poly</code> .
<code>add</code>	Function name.
<code>struct Poly *p1</code>	Pointer to the first polynomial.
<code>struct Poly *p2</code>	Pointer to the second polynomial.

Why return a pointer?

Because the result polynomial is created inside the `add()` function using `malloc`.

Heap memory remains available after the function ends.

So the function can safely return the address of the result.

24. Why the Result Is Created on the Heap

A wrong idea would be:

```
struct Poly sum;
return &sum;
```

This is wrong because `sum` is a local variable.

A local variable disappears when the function ends.

So returning its address is unsafe.

Correct idea:

```
struct Poly *sum;
sum = (struct Poly *) malloc(sizeof(struct Poly));
return sum;
```

This creates `sum` on the heap.

Heap memory remains available after the function returns.

25. Result Array Size

When adding two polynomials, we do not know the final number of terms at the beginning.

Worst case:

No exponents match.

Example:

```
P1 = 5x^4 + 2x^2  
P2 = 7x^3 + 9x^1
```

No exponents are the same.

So every term from both polynomials appears in the result.

If:

```
P1 has 2 terms  
P2 has 2 terms
```

Then the result may need:

```
2 + 2 = 4 terms
```

So we allocate:

```
maxTerms = p1->n + p2->n;  
sum->terms = (struct term *) malloc(maxTerms * sizeof(struct term));
```

But the actual result may be smaller.

Example:

```
5x^4 + 6x^4 = 11x^4
```

Two input terms become one output term.

That is why we use `k` to count the actual number of result terms.

At the end:

```
sum->n = k;
```

Do not write:

```
sum->n = p1->n + p2->n;
```

That would store the maximum possible count, not the actual count.

26. The Three Index Variables: `i`, `j`, and `k`

Polynomial addition uses three index variables:

```
int i = 0, j = 0, k = 0;
```

Meaning:

Variable	Meaning
<code>i</code>	current term in the first polynomial <code>p1</code>
<code>j</code>	current term in the second polynomial <code>p2</code>
<code>k</code>	current write position in the result polynomial <code>sum</code>

Beginner mental model:

```
i = finger pointing at p1  
j = finger pointing at p2  
k = finger pointing at the result
```

At the beginning:

```
i = 0  
j = 0  
k = 0
```

That means:


```
i points to p1->terms[0]
j points to p2->terms[0]
k points to sum->terms[0]
```

After copying or adding a term, the correct index moves forward.

27. Main Loop of Addition

The main loop is:

```
while (i < p1->n && j < p2->n) {
    // compare terms
}
```

Meaning:

Continue while both polynomials still have terms left.

The loop stops when either polynomial finishes.

After that, the remaining terms from the other polynomial are copied as leftovers.

28. Merge Rule 1: First Exponent Is Larger

Code:

```
if (p1->terms[i].exp > p2->terms[j].exp) {
    sum->terms[k++] = p1->terms[i++];
}
```

Meaning:

If the current term in `p1` has a larger exponent, it should appear first in the result.

So we copy the `p1` term.

Then we move:

```
i forward  
k forward
```

Example:

```
p1 term = 5x^4  
p2 term = 3x^2
```

Compare exponents:

```
4 > 2
```

So copy:

```
5x^4
```

29. Merge Rule 2: Second Exponent Is Larger

Code:

```
else if (p1->terms[i].exp < p2->terms[j].exp) {  
    sum->terms[k++] = p2->terms[j++];  
}
```

Meaning:

If the current term in `p2` has a larger exponent, it should appear first in the result.

So we copy the `p2` term.

Then we move:

```
j forward  
k forward
```

Example:

```
p1 term = 2x^2  
p2 term = 5x^3
```

Compare exponents:

```
2 < 3
```

So copy:

```
5x^3
```

30. Merge Rule 3: Exponents Are Equal

Code:

```
else {  
    int coefficientSum = p1->terms[i].coeff + p2->terms[j].coeff;  
  
    if (coefficientSum != 0) {  
        sum->terms[k].coeff = coefficientSum;  
        sum->terms[k].exp = p1->terms[i].exp;  
        k++;  
    }  
  
    i++;  
    j++;  
}
```

Meaning:

If the exponents are equal, the terms are like terms.

So we add the coefficients.

The exponent stays the same.

Example:

```
p1 term = 5x^4  
p2 term = 6x^4
```

Compare exponents:

```
4 == 4
```

Add coefficients:

```
5 + 6 = 11
```

Store:

```
11x^4
```

Then move:

```
i forward  
j forward  
k forward
```

Why move both `i` and `j`?

Because one term from each polynomial was used.

Why move `k`?

Because one result term was written.

31. Why Check `coefficientSum != 0`

Sometimes adding two coefficients gives zero.

Example:

```
5x^3 + (-5x^3) = 0x^3
```

A term with coefficient zero should usually not be stored.

So this code skips zero-result terms:

```
if (coefficientSum != 0) {
    sum->terms[k].coeff = coefficientSum;
    sum->terms[k].exp = p1->terms[i].exp;
    k++;
}
```

If the sum is zero, we do not store that term, and `k` does not increase.

This keeps the result cleaner.

32. Copying Leftover Terms

The main loop runs only while both polynomials still have terms.

```
while (i < p1->n && j < p2->n)
```

If one polynomial ends first, the other polynomial may still have remaining terms.

These remaining terms are called leftovers.

They must be copied into the result.

Why?

Because if a term exists only in one polynomial, it stays unchanged in the sum.

Copy leftovers from `p1`

```
while (i < p1->n) {
    sum->terms[k++] = p1->terms[i++];
}
```

Copy leftovers from `p2`

```
while (j < p2->n) {
    sum->terms[k++] = p2->terms[j++];
}
```

33. Full `add()` Function

```
struct Poly *add(struct Poly *p1, struct Poly *p2) {
    int i = 0, j = 0, k = 0;
    int maxTerms;
    struct Poly *sum;

    sum = (struct Poly *) malloc(sizeof(struct Poly));

    if (sum == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }

    maxTerms = p1->n + p2->n;

    sum->terms = (struct term *) malloc(maxTerms * sizeof(struct term));

    if (sum->terms == NULL && maxTerms > 0) {
        printf("Memory allocation failed.\n");
        free(sum);
        exit(1);
    }

    while (i < p1->n && j < p2->n) {
        if (p1->terms[i].exp > p2->terms[j].exp) {
            sum->terms[k++] = p1->terms[i++];
        }
        else if (p1->terms[i].exp < p2->terms[j].exp) {
            sum->terms[k++] = p2->terms[j++];
        }
        else {
            int coefficientSum = p1->terms[i].coeff + p2->terms[j].coeff;

            if (coefficientSum != 0) {
                sum->terms[k].coeff = coefficientSum;
                sum->terms[k].exp = p1->terms[i].exp;
                k++;
            }

            i++;
            j++;
        }
    }
}
```

```

while (i < p1->n) {
    sum->terms[k++] = p1->terms[i++];
}

while (j < p2->n) {
    sum->terms[k++] = p2->terms[j++];
}

sum->n = k;

return sum;
}

```

34. Line-by-Line Explanation of `add()`

Create variables

```
int i = 0, j = 0, k = 0;
```

These indexes control the merge.

```

i = current term in p1
j = current term in p2
k = current position in sum

```

Create result pointer

```
struct Poly *sum;
```

This will point to the result polynomial.

Allocate result structure

```
sum = (struct Poly *) malloc(sizeof(struct Poly));
```

This creates memory for one `struct Poly`.

Calculate maximum possible result size

```
maxTerms = p1->n + p2->n;
```

This is the worst-case number of result terms.

Allocate result term array

```
sum->terms = (struct term *) malloc(maxTerms * sizeof(struct term));
```

This creates memory for the result terms.

Compare terms

```
while (i < p1->n && j < p2->n)
```

The loop runs while both polynomials still have terms left.

Set actual result count

```
sum->n = k;
```

This stores the actual number of result terms.

Return result

```
return sum;
```

This sends the result polynomial back to `main()`.

35. The `main()` Function

The `main()` function connects everything.

It creates two polynomials, adds them, displays them, and frees memory.

Code:


```

int main() {
    struct Poly p1, p2;
    struct Poly *p3;

    printf("Enter first polynomial:\n");
    create(&p1);

    printf("\nEnter second polynomial:\n");
    create(&p2);

    p3 = add(&p1, &p2);

    printf("\nFirst Polynomial:\n");
    display(p1);

    printf("\nSecond Polynomial:\n");
    display(p2);

    printf("\nSum Polynomial:\n");
    display(*p3);

    free(p1.terms);
    free(p2.terms);
    free(p3->terms);
    free(p3);

    return 0;
}

```

36. Explanation of `main()`

Create two polynomial variables

```

struct Poly p1, p2;

```

This creates two normal polynomial structures.

Create a result pointer

```

struct Poly *p3;

```

This pointer will store the address of the result polynomial.

Create first polynomial

```
create(&p1);
```

This sends the address of `p1` to `create()`.

Create second polynomial

```
create(&p2);
```

This sends the address of `p2` to `create()`.

Add the polynomials

```
p3 = add(&p1, &p2);
```

This sends the addresses of `p1` and `p2` to `add()`.

The returned result pointer is stored in `p3`.

Display the polynomials

```
display(p1);  
display(p2);  
display(*p3);
```

Notice:

```
display(*p3);
```

Why `*p3`?

Because `display()` expects a normal structure value:

```
void display(struct Poly p)
```

But `p3` is a pointer.

So we write:

```
*p3
```

This means:

```
the actual polynomial that p3 points to
```

Free memory

```
free(p1.terms);  
free(p2.terms);  
free(p3->terms);  
free(p3);
```

This releases all memory created by `malloc`.

37. Full Lesson 12 C Program

```
#include <stdio.h>  
#include <stdlib.h>  
  
struct term {  
    int coeff;  
    int exp;  
};  
  
struct Poly {  
    int n;  
    struct term *terms;  
};  
  
void create(struct Poly *p) {  
    int i;  
  
    printf("Enter number of terms: ");  
    scanf("%d", &p->n);
```

```

p->terms = (struct term *) malloc(p->n * sizeof(struct term));

if (p->terms == NULL && p->n > 0) {
    printf("Memory allocation failed.\n");
    exit(1);
}

printf("Enter coefficient and exponent for each term in descending exponent
order:\n");

for (i = 0; i < p->n; i++) {
    scanf("%d %d", &p->terms[i].coeff, &p->terms[i].exp);
}
}

void display(struct Poly p) {
    int i;

    if (p.n == 0) {
        printf("0\n");
        return;
    }

    for (i = 0; i < p.n; i++) {
        printf("%dx^%d", p.terms[i].coeff, p.terms[i].exp);

        if (i < p.n - 1) {
            printf(" + ");
        }
    }

    printf("\n");
}

struct Poly *add(struct Poly *p1, struct Poly *p2) {
    int i = 0, j = 0, k = 0;
    int maxTerms;
    struct Poly *sum;

    sum = (struct Poly *) malloc(sizeof(struct Poly));

    if (sum == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }

    maxTerms = p1->n + p2->n;

```

```

sum->terms = (struct term *) malloc(maxTerms * sizeof(struct term));

if (sum->terms == NULL && maxTerms > 0) {
    printf("Memory allocation failed.\n");
    free(sum);
    exit(1);
}

while (i < p1->n && j < p2->n) {
    if (p1->terms[i].exp > p2->terms[j].exp) {
        sum->terms[k++] = p1->terms[i++];
    }
    else if (p1->terms[i].exp < p2->terms[j].exp) {
        sum->terms[k++] = p2->terms[j++];
    }
    else {
        int coefficientSum = p1->terms[i].coeff + p2->terms[j].coeff;

        if (coefficientSum != 0) {
            sum->terms[k].coeff = coefficientSum;
            sum->terms[k].exp = p1->terms[i].exp;
            k++;
        }

        i++;
        j++;
    }
}

while (i < p1->n) {
    sum->terms[k++] = p1->terms[i++];
}

while (j < p2->n) {
    sum->terms[k++] = p2->terms[j++];
}

sum->n = k;

return sum;
}

int main() {
    struct Poly p1, p2;
    struct Poly *p3;

    printf("Enter first polynomial:\n");
    create(&p1);

```

```

    printf("\nEnter second polynomial:\n");
    create(&p2);

    p3 = add(&p1, &p2);

    printf("\nFirst Polynomial:\n");
    display(p1);

    printf("\nSecond Polynomial:\n");
    display(p2);

    printf("\nSum Polynomial:\n");
    display(*p3);

    free(p1.terms);
    free(p2.terms);
    free(p3->terms);
    free(p3);

    return 0;
}

```

38. Example Input and Output

Input

```

Enter first polynomial:
Enter number of terms: 3
Enter coefficient and exponent for each term in descending exponent order:
5 4
2 2
5 0

Enter second polynomial:
Enter number of terms: 5
Enter coefficient and exponent for each term in descending exponent order:
6 4
5 3
9 2
2 1
3 0

```

This means:

$$\begin{aligned} P1 &= 5x^4 + 2x^2 + 5x^0 \\ P2 &= 6x^4 + 5x^3 + 9x^2 + 2x^1 + 3x^0 \end{aligned}$$

Output

```
First Polynomial:
5x^4 + 2x^2 + 5x^0

Second Polynomial:
6x^4 + 5x^3 + 9x^2 + 2x^1 + 3x^0

Sum Polynomial:
11x^4 + 5x^3 + 11x^2 + 2x^1 + 8x^0
```

Usually, we write the result as:

$$11x^4 + 5x^3 + 11x^2 + 2x + 8$$

But the program prints every term in the same format:

```
coefficient x^ exponent
```

So it prints:

```
2x^1
8x^0
```

39. Dry Run 1: Main Addition Example

Let:

$$\begin{aligned} P1 &= 5x^4 + 2x^2 + 5x^0 \\ P2 &= 6x^4 + 5x^3 + 9x^2 + 2x^1 + 3x^0 \end{aligned}$$

Stored form:

P1:

Index	Coeff	Exp
0	5	4
1	2	2
2	5	0

P2:

Index	Coeff	Exp
0	6	4
1	5	3
2	9	2
3	2	1
4	3	0

Initial values:

```
i = 0
j = 0
k = 0
```

Step 1

Current terms:

```
P1[i] = 5x^4
P2[j] = 6x^4
```

Compare exponents:

```
4 == 4
```

The exponents are equal.

Add coefficients:

```
5 + 6 = 11
```

Store:

$\text{sum}[0] = 11x^4$

Move:

$i = 1$
 $j = 1$
 $k = 1$

Current result:

$11x^4$

Step 2

Current terms:

$P1[i] = 2x^2$
 $P2[j] = 5x^3$

Compare exponents:

$2 < 3$

The second polynomial has the larger exponent.

Copy:

$5x^3$

Move:

$j = 2$
 $k = 2$

i stays the same:

$i = 1$

Current result:

$11x^4 + 5x^3$

Step 3

Current terms:

$P1[i] = 2x^2$
 $P2[j] = 9x^2$

Compare exponents:

$2 == 2$

The exponents are equal.

Add coefficients:

$2 + 9 = 11$

Store:

$11x^2$

Move:

$i = 2$
 $j = 3$
 $k = 3$

Current result:

$$11x^4 + 5x^3 + 11x^2$$

Step 4

Current terms:

$$\begin{aligned} P1[i] &= 5x^0 \\ P2[j] &= 2x^1 \end{aligned}$$

Compare exponents:

$$0 < 1$$

The second polynomial has the larger exponent.

Copy:

$$2x^1$$

Move:

$$\begin{aligned} j &= 4 \\ k &= 4 \end{aligned}$$

i stays the same:

$$i = 2$$

Current result:

$$11x^4 + 5x^3 + 11x^2 + 2x^1$$

Step 5

Current terms:

```
P1[i] = 5x^0  
P2[j] = 3x^0
```

Compare exponents:

```
0 == 0
```

The exponents are equal.

Add coefficients:

```
5 + 3 = 8
```

Store:

```
8x^0
```

Move:

```
i = 3  
j = 5  
k = 5
```

Now both polynomials are finished.

Final result:

```
11x^4 + 5x^3 + 11x^2 + 2x^1 + 8x^0
```

40. Dry Run 2: Overlap Case

Use this example:

```
P1 = 1x^3 + 1x^2 + 1x^1  
P2 = 5x^2 + 5x^1 + 5x^0
```

Input:

Enter first polynomial:

3

1 3

1 2

1 1

Enter second polynomial:

3

5 2

5 1

5 0

Stored form:

P1:

Index	Coeff	Exp
-------	-------	-----

0	1	3
---	---	---

1	1	2
---	---	---

2	1	1
---	---	---

P2:

Index	Coeff	Exp
-------	-------	-----

0	5	2
---	---	---

1	5	1
---	---	---

2	5	0
---	---	---

Initial values:

i = 0

j = 0

k = 0

Step 1

Current terms:

P1[i] = $1x^3$

P2[j] = $5x^2$

Compare exponents:

$$3 > 2$$

Copy the first polynomial term:

$$1x^3$$

Move:

$$\begin{aligned} i &= 1 \\ k &= 1 \end{aligned}$$

Current result:

$$1x^3$$

Step 2

Current terms:

$$\begin{aligned} P1[i] &= 1x^2 \\ P2[j] &= 5x^2 \end{aligned}$$

Compare exponents:

$$2 == 2$$

Add coefficients:

$$1 + 5 = 6$$

Store:

$$6x^2$$

Move:

```
i = 2  
j = 1  
k = 2
```

Current result:

```
1x^3 + 6x^2
```

Step 3

Current terms:

```
P1[i] = 1x^1  
P2[j] = 5x^1
```

Compare exponents:

```
1 == 1
```

Add coefficients:

```
1 + 5 = 6
```

Store:

```
6x^1
```

Move:

```
i = 3  
j = 2  
k = 3
```

Current result:

$$1x^3 + 6x^2 + 6x^1$$

Step 4: Copy Leftover

Now **P1** is finished.

But **P2** still has:

$$5x^0$$

Copy it:

$$5x^0$$

Move:

$$\begin{aligned} j &= 3 \\ k &= 4 \end{aligned}$$

Final result:

$$1x^3 + 6x^2 + 6x^1 + 5x^0$$

41. Dry Run 3: Coefficients Cancel to Zero

Let:

$$\begin{aligned} P1 &= 5x^3 + 2x^1 \\ P2 &= -5x^3 + 4x^0 \end{aligned}$$

Stored form:

$$\begin{array}{l} P1: \\ 5 \ 3 \\ 2 \ 1 \end{array}$$

P2:
-5 3
4 0

Step 1:

$$5x^3 + (-5x^3)$$

Exponents match:

$$3 == 3$$

Add coefficients:

$$5 + (-5) = 0$$

Because the coefficient sum is zero, do not store this term.

Result is still empty.

Step 2:

Current terms:

$$\begin{aligned} P1 &= 2x^1 \\ P2 &= 4x^0 \end{aligned}$$

Compare exponents:

$$1 > 0$$

Copy:

$$2x^1$$

Step 3:

Copy leftover:

$4x^0$

Final result:

$2x^1 + 4x^0$

This shows why the check `coefficientSum != 0` is useful.

42. Time and Space Complexity

`create()` Complexity

The `create()` function reads each term once.

If the polynomial has `n` terms, the loop runs `n` times.

So:

```
Time complexity = O(n)
Space complexity = O(n)
```

Why space is `O(n)` ?

Because the program creates an array of `n` terms.

`display()` Complexity

The `display()` function prints each term once.

If the polynomial has `n` terms, the loop runs `n` times.

So:

```
Time complexity = O(n)
Space complexity = O(1)
```

Why space is `O(1)` ?

Because it only uses a few extra variables, such as `i`.

`add()` Complexity

Let:

```
p1 has n terms  
p2 has m terms
```

The `add()` function scans both term arrays once.

Each term is processed at most one time.

So:

```
Time complexity =  $O(n + m)$ 
```

The result array is allocated for the worst case:

```
n + m terms
```

So:

```
Space complexity =  $O(n + m)$ 
```

43. Complexity Table

Operation	Time Complexity	Space Complexity	Reason
<code>create()</code>	$O(n)$	$O(n)$	Reads and stores <code>n</code> terms.
<code>display()</code>	$O(n)$	$O(1)$	Prints each term once.
<code>add()</code>	$O(n + m)$	$O(n + m)$	Merges both term arrays once.

44. Important Assumptions

The program assumes:

1. Terms are entered in descending exponent order.
2. Each polynomial should not contain duplicate exponents inside itself.
3. `n` correctly matches the number of terms entered.
4. Memory allocation succeeds.
5. Coefficients and exponents are integers.

Why duplicate exponents inside one polynomial are a problem

Example:

```
P1 = 3x^2 + 5x^2
```

This polynomial should first be simplified to:

```
8x^2
```

The addition function combines terms between two polynomials.

It does not first simplify duplicate terms inside one polynomial.

So the user should enter each polynomial in simplified form.

45. Common Beginner Mistakes

Mistake	Why it is wrong	Correction
Thinking <code>n</code> means highest exponent	<code>n</code> stores the number of terms.	Use <code>n</code> only as term count.
Entering terms in random order	The merge algorithm moves forward only.	Enter terms in descending exponent order.
Setting <code>sum->n = p1->n + p2->n</code>	That is only the maximum possible size.	Use <code>sum->n = k</code> .
Forgetting <code>free()</code>	Causes memory leaks.	Free every block created with <code>malloc</code> .

Mistake	Why it is wrong	Correction
Using <code>.</code> instead of <code>-></code> for pointers	Pointer fields need <code>-></code> .	Use <code>-></code> when variable is a pointer.
Returning address of a local variable	Local variables disappear after the function ends.	Allocate result using <code>malloc</code> .
Printing <code>+</code> after the last term	Output looks wrong.	Print <code>+</code> only between terms.
Not checking memory allocation	Program may crash if memory allocation fails.	Check if pointer is <code>NULL</code> .
Storing zero coefficient terms	Wastes space and makes output messy.	Skip terms whose coefficient becomes zero.

46. Quick Summary of Lesson 12

Lesson 12 builds a complete C program for polynomial addition.

The polynomial is stored as an array of terms.

Each term stores:

```
coefficient, exponent
```

The full polynomial structure stores:

```
number of terms, pointer to terms array
```

The program uses:

```
create() → read polynomial
display() → print polynomial
add() → add two polynomials
main() → connect everything
```

The addition algorithm uses three indexes:

```
i → current term in first polynomial
j → current term in second polynomial
k → current result position
```

The merge rules are:

```
If p1 exponent is larger:
    copy p1 term
```

```
If p2 exponent is larger:
    copy p2 term
```

```
If exponents are equal:
    add coefficients
```

```
If one polynomial ends:
    copy leftovers
```

The final number of result terms is:

```
sum->n = k;
```

not:

```
sum->n = p1->n + p2->n;
```

Because `p1->n + p2->n` is only the maximum possible number of terms.

47. Final Lesson 12 Checkpoint

By the end of Lesson 12, you should be able to:

- explain how a polynomial is stored using structures,
- explain the meaning of `coeff` and `exp`,
- explain why `n` means number of terms,
- write the `struct term` and `struct Poly` definitions,
- use `malloc` to allocate the term array,
- use `free` to release memory,
- explain the difference between `.` and `->`,
- write the `create()` function,

- write the `display()` function,
- write the `add()` function,
- explain why the result array size is `p1->n + p2->n`,
- explain why the final result count is `k`,
- dry run the movement of `i`, `j`, and `k`,
- state the time and space complexity of each function.

If you can explain why `i`, `j`, and `k` move in each case, then you understand the core of Lesson 12.